

**REPÚBLICA BOLIVARIANA DE VENEZUELA
MINISTERIO DEL PODER POPULAR PARA LA EDUCACIÓN UNIVERSITARIA
UNIVERSIDAD NACIONAL EXPERIMENTAL DE LAS TELECOMUNICACIONES E
INFORMÁTICA**

**SISTEMA CRUD DE PELÍCULAS CON ANGULAR, NODE.JS Y
MONGODB**

Autor:
Miguel Figuera
C.I.: 23.558.789

La Victoria, julio, 2026

ÍNDICE

Introducción

Etapas 1. Montaje del Monorepo y Configuración del Entorno

Etapas 2. Backend Node.js: Conexión a MongoDB, Seed y API CRUD

Etapas 3. Scripts de Seed Multiplataforma e Integración de Tailwind CSS

Etapas 4. Frontend Angular: Componentes UI y Página de Películas

Etapas 5. Orquestación de la Ejecución y Arquitectura de Tres Capas

Etapas 6. Publicación del Repositorio en GitHub

Etapas 7. Verificación End-to-End con Playwright

Etapas 8. Modo Oscuro por Defecto y Cierre del Proyecto

Conclusión

Referencias

Anexos

INTRODUCCIÓN

El presente documento describe el proceso metodológico seguido para el desarrollo de un sistema de gestión de películas con operaciones CRUD (crear, leer, actualizar y eliminar), elaborado como evaluación del módulo Informática — Bases de Datos del Trayecto III de la Universidad Nacional Experimental de las Telecomunicaciones e Informática (UNETI), extensión La Victoria. La aplicación integra tres capas tecnológicas claramente diferenciadas: una interfaz de usuario construida con Angular 19 y Tailwind CSS, una interfaz de programación de aplicaciones (API) REST desarrollada con Node.js y Express, y una base de datos documental MongoDB con cinco colecciones pobladas mediante scripts de inicialización.

El trabajo se organizó en ocho etapas consecutivas, documentadas mediante doce capturas de pantalla tomadas durante las sesiones de desarrollo asistido en el editor Cursor. Estas evidencias registran el montaje del monorepo con espacios de trabajo de Yarn, la construcción del backend con su conexión a MongoDB y su enrutador CRUD genérico, la creación de scripts de poblado de datos para Linux y Windows, la implementación del frontend Angular con componentes reutilizables, la orquestación de la ejecución simultánea de ambos servidores, la publicación del código fuente en GitHub, la verificación integral del sistema con pruebas end-to-end de Playwright y, finalmente, la activación del modo oscuro como tema visual de la aplicación.

Este documento se redacta siguiendo las normas de la American Psychological Association en su séptima edición, con la única excepción de que los párrafos del cuerpo se presentan justificados en lugar de alineados a la izquierda, conforme a los lineamientos internos de la asignatura. Cada etapa incluye los fragmentos de código fuente más representativos del

repositorio, de modo que el lector pueda comprender tanto las decisiones técnicas tomadas como reproducir el procedimiento completo en un entorno propio.

ETAPA 1

Montaje del Monorepo y Configuración del Entorno

La primera etapa consistió en verificar las herramientas del entorno de desarrollo y montar la estructura general del proyecto. Se comprobó la disponibilidad de Node.js v25.2.0 y Yarn 1.22.22 en el equipo de trabajo, y se constató que el espacio de trabajo se encontraba vacío. A partir de allí se definió el plan de construcción en cuatro frentes: un backend Node.js con Express y MongoDB con operaciones CRUD sobre cinco colecciones, un frontend Angular con Tailwind CSS y componentes de estilo shadcn/ui, scripts de poblado de datos (seed) para Linux y Windows, y un archivo README con las instrucciones de ejecución.

El andamiaje del frontend se generó con Angular CLI en su versión 19, mediante el comando `npx @angular/cli@19 new frontend` con las opciones `--style=css`, `--routing=true`, `--ssr=false`, `--skip-git` y `--package-manager=yarn`, lo que produjo una aplicación moderna basada en componentes standalone. Paralelamente se creó el archivo `package.json` de la raíz, que declara los espacios de trabajo (workspaces) backend y frontend de Yarn, y define los scripts de orquestación del proyecto completo.

package.json (raíz del monorepo)

```
{
  "name": "cine-uneti",
  "version": "1.0.0",
  "private": true,
  "description": "Sistema CRUD de películas con Angular, Node.js y MongoDB",
  "workspaces": ["backend", "frontend"],
  "scripts": {
    "install:all": "yarn install",
    "postinstall": "node scripts/run-seed.js",
    "start": "concurrently -n api,web -c blue,green \\\n    \\\"yarn workspace backend start\\\" \\\"yarn workspace frontend start\\\"",
    "start:backend": "yarn workspace backend start",
    "start:frontend": "yarn workspace frontend start",
    "seed": "node scripts/run-seed.js"
  },
}
```

```
"devDependencies": { "concurrently": "^9.1.2" },  
"engines": { "node": ">=18" }  
}
```

La configuración sensible se externalizó en variables de entorno. El archivo `.env.example` documenta los tres valores necesarios para ejecutar el sistema: la URI de conexión a MongoDB, el nombre de la base de datos y el puerto de la API. La figura 1 muestra este momento inicial del proceso: la verificación de versiones, la lista de tareas planificadas, la ejecución de Angular CLI y la edición de los `package.json` de la raíz y del backend.

.env.example

```
MONGODB_URI=mongodb://localhost:27017  
MONGODB_DB=cine_uneti  
PORT=3000
```

```

me hare cargo de eso luego.

Revisando el workspace y montando la estructura del proyecto (Angular + Node.js + MongoDB).

Globbed "**/*" in .
Found 0 files.

$ ls -la /home/chepino/repos/Uneti-related/informatica-basededatos 2>/dev/null; node --version; yarn --version
2>/dev/null; ng version 2>/dev/null | head -5 1.0s
_ 3 output lines hidden · ctrl+o to expand
v25.2.0
1.22.22

El workspace está vacío. Voy a montar el proyecto completo: Angular con Tailwind, backend Node.js con MongoDB
y scripts de seed.

To-do Working on 4 to-dos
o Crear backend Node.js + Express + MongoDB con CRUD (5 colecciones)
o Crear frontend Angular + Tailwind + componentes shadcn-style
o Scripts bash/PowerShell seed + postinstall con yarn
o README con instrucciones de ejecución

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && npx -y @angular/cli@19 new frontend
--style=css --routing=true --ssr=false --skip-git --package-manager=yarn --defaults 2>&1 42s
_ 33 output lines hidden · ctrl+o to expand
npm notice To update run: npm install -g npm@11.18.0
npm notice

Edited package.json +24 -1
+ {
+   "name": "cine-uneti",
+   "version": "1.0.0",
+   "private": true,
+   "description": "Sistema CRUD de películas con Angular, Node.js y MongoDB",
+   "workspaces": [
+     "backend",
+     "frontend"
+   ],
+   "scripts": {
+     "install:all": "yarn install",
+   },
+   "truncated (13 more lines) · ctrl+r to review

Edited package.json +18 -1
+ {
+   "name": "backend",
+   "version": "1.0.0",
+   "private": true,
+   "description": "API REST Node.js + MongoDB",
+   "main": "src/index.js",
+   "scripts": {
+     "start": "node src/index.js",
+     "dev": "node --watch src/index.js",
+     "seed": "node src/seed/seed.js"
+   },
+   "truncated (7 more lines) · ctrl+r to review

Edited .env.example +3 -1
+ MONGODB_URI=mongodb://localhost:27017
+ MONGODB_DB=cine_uneti
+ PORT=3000

```

Figura 1. Montaje inicial del monorepo: verificación de herramientas, plan de trabajo, creación del frontend con Angular CLI 19 y edición de los package.json.

ETAPA 2

Backend Node.js: Conexión a MongoDB, Seed y API CRUD

La segunda etapa construyó la capa de datos y la API REST. El módulo `backend/src/config/db.js` encapsula la conexión a MongoDB con el patrón de instancia única: la función `connectDB()` abre el cliente una sola vez, verifica la disponibilidad del servidor con el comando `ping` y expone la base de datos mediante `getDB()`. Los valores de conexión provienen de las variables de entorno definidas en la etapa anterior, con valores por defecto para el entorno local.

backend/src/config/db.js (fragmento)

```
const { MongoClient } = require('mongodb');

const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017';
const dbName = process.env.MONGODB_DB || 'cine_uneti';

let client;
let db;
let isConnected = false;

async function connectDB() {
  if (isConnected && db) return db;

  client = new MongoClient(uri, { serverSelectionTimeoutMS: 4000 });
  await client.connect();
  await client.db(dbName).command({ ping: 1 });
  db = client.db(dbName);
  isConnected = true;
  console.log(`Conectado a MongoDB: ${dbName}`);
  return db;
}
```

El poblado inicial de la base de datos se definió en `backend/src/seed/seed.js`, que inserta documentos en las cinco colecciones exigidas por la evaluación: películas (título, año, género, duración y calificación), directores, generos, actores y resenas, cada una con al menos cuatro campos y cinco documentos. El script elimina los datos previos con `deleteMany` y los reinserta con `insertMany`, de modo que la operación sea repetible sin duplicar registros.

La pieza central de la API es el enrutador CRUD genérico de `backend/src/routes/crudRouter.js`. En lugar de escribir cinco juegos de rutas casi idénticos, se definió un único conjunto de rutas parametrizado por el nombre de la colección. El método `router.param` valida que la colección solicitada pertenezca a la lista blanca `ALLOWED_COLLECTIONS` y rechaza cualquier otra con un error 400, lo que evita el acceso arbitrario a colecciones de la base de datos. Sobre esa base se implementaron las operaciones GET (listar y obtener por identificador), POST (crear), PUT (actualizar) y DELETE (eliminar), además de la consulta sencilla requerida: filtrar películas por género.

backend/src/routes/crudRouter.js (fragmento)

```
const ALLOWED_COLLECTIONS =
  ['peliculas', 'directores', 'generos', 'actores', 'resenas'];

function createCrudRouter() {
  const router = express.Router({ mergeParams: true });

  router.param('collection', (req, res, next, collection) => {
    if (!ALLOWED_COLLECTIONS.includes(collection)) {
      return res.status(400).json({
        error: `Colección no permitida. Use: ${ALLOWED_COLLECTIONS.join(', ')}`,
      });
    }
    req.collectionName = collection;
    next();
  });

  // READ - Consulta sencilla: películas por género
  router.get('/:collection/consulta/genero/:genero', async (req, res) => {
    const pattern = buildSpanishInsensitivePattern(req.params.genero);
    const docs = await getDB()
      .collection('peliculas')
      .find({ genero: { $regex: pattern, $options: 'i' } })
      .toArray();
    res.json({ genero: req.params.genero, total: docs.length, peliculas: docs });
  });

  // CREATE
  router.post('/:collection', async (req, res) => {
    const result = await getDB()
      .collection(req.collectionName)
      .insertOne(req.body);
    const created = await getDB()
      .collection(req.collectionName)
      .findOne({ _id: result.insertedId });
  });
}
```

```
    res.status(201).json(created);  
  });  
  // ... GET por id, PUT y DELETE siguen el mismo patrón  
  return router;  
}
```

Finalmente, `backend/src/index.js` integra las piezas: habilita CORS para permitir las peticiones desde el frontend en el puerto 4200, registra el middleware `express.json()` para interpretar cuerpos JSON, expone el punto de verificación `/api/health` y monta el enrutador CRUD bajo el prefijo `/api`. La figura 2 documenta la escritura de estos archivos durante la sesión de desarrollo.

```

+ MONGODB_URI=mongodb://localhost:27017
+ MONGODB_DB=cine_uneti
+ PORT=3000

Edited .env.example +3/-1 d chats
-
+ MONGODB_URI=mongodb://localhost:27017
+ MONGODB_DB=cine_uneti
+ PORT=3000

Edited db.js +34
+ const { MongoClient } = require('mongodb');
+ require('dotenv').config({ path: require('path').join(__dirname, '../.env') });
+
+ const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017';
+ const dbName = process.env.MONGODB_DB || 'cine_uneti';
+
+ let client;
+ let db;
+
+ async function connectDB() {
+   if (db) return db;
+
+   ... truncated (23 more lines) · ctrl+r to review

Edited seed.js +73
+ const { MongoClient } = require('mongodb');
+ const path = require('path');
+ require('dotenv').config({ path: path.join(__dirname, '../.env') });
+
+ const uri = process.env.MONGODB_URI || 'mongodb://localhost:27017';
+ const dbName = process.env.MONGODB_DB || 'cine_uneti';
+
+ const seedData = {
+   peliculas: [
+     { titulo: 'El Padrino', anio: 1972, genero: 'Drama', duracionMinutos: 175, calificacion: 9.2 },
+     { titulo: 'Pulp Fiction', anio: 1994, genero: 'Crimen', duracionMinutos: 154, calificacion: 8.9 },
+     { titulo: 'El Señor de los Anillos', anio: 2001, genero: 'Fantasia', duracionMinutos: 178,
+       calificacion: 8.8 },
+   ],
+   ... truncated (62 more lines) · ctrl+r to review

Edited crudRouter.js +111
+ const express = require('express');
+ const { ObjectId } = require('mongodb');
+ const { getDB } = require('../config/db');
+
+ const ALLOWED_COLLECTIONS = ['peliculas', 'directores', 'generos', 'actores', 'resenas'];
+
+ function createCrudRouter() {
+   const router = express.Router({ mergeParams: true });
+
+   router.param('collection', (req, res, next, collection) => {
+     if (!ALLOWED_COLLECTIONS.includes(collection)) {
+       return res.status(400).json({
+         ... truncated (100 more lines) · ctrl+r to review

Edited index.js +37
+ const express = require('express');
+ const cors = require('cors');
+ const path = require('path');
+ require('dotenv').config({ path: path.join(__dirname, '../.env') });
+
+ const { connectDB } = require('../config/db');
+ const { createCrudRouter, ALLOWED_COLLECTIONS } = require('../routes/crudRouter');
+
+ const app = express();
+ const PORT = process.env.PORT || 3000;

```

Figura 2. Construcción del backend: conexión a MongoDB (db.js), seed de las cinco colecciones, enrutador CRUD genérico e index.js de Express.

ETAPA 3

Scripts de Seed Multiplataforma e Integración de Tailwind CSS

La tercera etapa resolvió dos necesidades de infraestructura. La primera fue garantizar que la base de datos pudiera poblarse en cualquier sistema operativo: se escribieron `scripts/seed-db.sh` para Linux y macOS (bash) y `scripts/seed-db.ps1` para Windows (PowerShell), ambos con la misma lógica de localizar la raíz del proyecto y ejecutar el seed de Node.js. Sobre ellos se creó `scripts/run-seed.js`, un orquestador que detecta el sistema operativo en tiempo de ejecución, invoca el script correspondiente y, si este falla, recurre directamente al seed de Node.js como mecanismo de respaldo.

scripts/run-seed.js (fragmento)

```
const rootDir = path.join(__dirname, '..');
const isWindows = process.platform === 'win32';

if (process.env.SKIP_SEED === '1') {
  console.log('SKIP_SEED=1 - seed omitido.\n');
  process.exit(0);
}

if (isWindows) {
  const psScript = path.join(__dirname, 'seed-db.ps1');
  const result = spawnSync(
    'powershell',
    ['-ExecutionPolicy', 'Bypass', '-File', psScript],
    { cwd: rootDir, stdio: 'inherit' }
  );
  if (result.status !== 0) runNodeSeed();
} else {
  const shScript = path.join(__dirname, 'seed-db.sh');
  fs.chmodSync(shScript, '755');
  const result = spawnSync('bash', [shScript], { cwd: rootDir, stdio: 'inherit' });
  if (result.status !== 0) runNodeSeed();
}
```

Este orquestador se registró como script postinstall del `package.json` raíz, de manera que al ejecutar `yarn install` la base de datos quede creada y poblada automáticamente con las cinco colecciones, sin pasos manuales adicionales. Si MongoDB no está activo en ese momento, el

proceso lo advierte y continúa sin interrumpir la instalación; el usuario puede completar el poblado más tarde con yarn seed.

La segunda necesidad fue integrar Tailwind CSS en el proyecto Angular. Durante la instalación se detectó que Tailwind v4 cambió su modelo de configuración y presentaba fricciones con Angular 19, por lo que se tomó la decisión técnica de fijar la versión 3 (tailwindcss@3) junto con PostCSS y Autoprefixer, generando los archivos tailwind.config.js y postcss.config.js tradicionales. La figura 3 muestra la escritura de los tres scripts de seed y el momento exacto de esa corrección de versiones.

```

Edited seed-db.sh +79
+#!/usr/bin/env bash
+ # Pobra la base de datos MongoDB con las 5 colecciones del proyecto.
+ set -euo pipefail
+
+ SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
+ ROOT_DIR="$(cd "$SCRIPT_DIR/.." && pwd)"
+
+ echo "=====
+ echo " Seed MongoDB - Cine UNETI (Linux)"
+ echo "=====
+
+ if ! command -v mongosh &>/dev/null && ! command -v mongo &>/dev/null; then
+   _truncated (68 more lines) · ctrl+r to review
+
Edited seed-db.ps1 +86
+ # Pobra la base de datos MongoDB con las 5 colecciones del proyecto.
+ $ErrorActionPreference = "Stop"
+
+ $ScriptDir = Split-Path -Parent $MyInvocation.MyCommand.Path
+ $RootDir = Split-Path -Parent $ScriptDir
+
+ Write-Host "=====
+ Write-Host " Seed MongoDB - Cine UNETI (Windows)"
+ Write-Host "=====
+
+ $MongoUri = if ($env:MONGODB_URI) { $env:MONGODB_URI } else { "mongodb://localhost:27017" }
+ $DbName = if ($env:MONGODB_DB) { $env:MONGODB_DB } else { "cine_uneti" }
+
+ _truncated (75 more lines) · ctrl+r to review
+
Edited run-seed.js +49
+/**
+ * Ejecuta el seed de la base de datos después de yarn install.
+ * Detecta el SO y usa el script correspondiente, con fallback a Node.js.
+ */
+const { spawnSync } = require('child_process');
+const path = require('path');
+const fs = require('fs');
+
+const rootDir = path.join(__dirname, '..');
+const isWindows = process.platform === 'win32';
+
+console.log('\n--- Postinstall: poblando base de datos MongoDB ---\n');
+
+ _truncated (38 more lines) · ctrl+r to review
+
$ cd /home/chepino/repos/Uneti-related/informatica-basededatos/frontend && yarn add -D tailwindcss postcss autoprefixer && npx tailwindcss init -p 2>&1 exit 1 · 15s
_ 593 output lines hidden · ctrl+o to expand
npm error could not determine executable to run
npm error A complete log of this run can be found in: /tmp/cursor-sandbox-cache/68e920f49c146b97c3178b14e7ab...
+
+ Read 2 files
+ Read frontend/src/app/app.config.ts
+ Read frontend/angular.json
+
Tailwind v4 cambió la configuración; usaré v3 para integrarlo mejor con Angular.
$ cd /home/chepino/repos/Uneti-related/informatica-basededatos/frontend && yarn remove tailwindcss && yarn add -D tailwindcss@3 postcss autoprefixer && yarn tailwindcss init -p 2>&1 5.5s
_ 55 output lines hidden · ctrl+o to expand
Created PostCSS config file: postcss.config.js
Done in 0.15s.
+
+ Edited tailwind.config.js +46 -1
+
+ /** @type {import('tailwindcss').Config} */
+ module.exports = {

```

Figura 3. Scripts de seed para Linux (bash) y Windows (PowerShell), orquestador run-seed.js e integración de Tailwind CSS 3 en Angular.

ETAPA 4

Frontend Angular: Componentes UI y Página de Películas

La cuarta etapa construyó la interfaz de usuario. Siguiendo la filosofía de shadcn/ui, se crearon componentes de interfaz propios y reutilizables en `frontend/src/app/components/ui/`: un botón (`ui-button`), un campo de texto (`ui-input`) que implementa la interfaz `ControlValueAccessor` de Angular para integrarse con los formularios reactivos, y una familia de tarjetas (`ui-card`, con encabezado, título, descripción y contenido). Todos son componentes standalone estilizados con clases utilitarias de Tailwind, lo que evita dependencias de bibliotecas de componentes externas.

La comunicación con la API se concentró en el servicio `frontend/src/app/services/pelicula.service.ts`, que define la interfaz TypeScript `Pelicula` y expone métodos `Observable` para cada operación CRUD y para la consulta por género. El servicio incluye además un manejo de errores específico: cuando la API responde 503 porque MongoDB no está disponible, se lanza el error tipado `MongoUnavailableError`, que la interfaz traduce en un aviso claro para el usuario en lugar de un fallo genérico.

frontend/src/app/services/pelicula.service.ts (fragmento)

```
export interface Pelicula {
  _id?: string;
  titulo: string;
  anio?: number;
  genero?: string;
  duracionMinutos?: number;
  calificacion?: number;
}

@Injectable({ providedIn: 'root' })
export class PeliculaService {
  private readonly baseUrl = `${environment.apiUrl}/peliculas`;

  constructor(private http: HttpClient) {}

  listar(): Observable<Pelicula[]> {
    return this.http.get<Pelicula[]>(this.baseUrl)
  }
}
```

```

        .pipe(catchError(this.handleError));
    }

    crear(pelicula: Partial<Película>): Observable<Película> {
        return this.http.post<Película>(this.baseUrl, pelicula)
            .pipe(catchError(this.handleError));
    }

    eliminar(id: string): Observable<{ message: string }> {
        return this.http.delete<{ message: string }>(`${this.baseUrl}/${id}`)
            .pipe(catchError(this.handleError));
    }

    consultarPorGenero(genero: string) {
        return this.http.get(
            `${this.baseUrl}/consulta/genero/${encodeURIComponent(genero)}`
        ).pipe(catchError(this.handleError));
    }
}

```

La página principal, `frontend/src/app/pages/peliculas/peliculas.component.ts`, consume ese servicio mediante formularios reactivos: un `FormGroup` con validadores exige un título de al menos dos caracteres y precarga el año actual y el género Drama. Desde esta vista el usuario puede añadir películas a la lista, verlas con sus datos del seed, eliminarlas y ejecutar la consulta por género contra MongoDB. La figura 4 registra la creación de los componentes de interfaz y de esta página.

frontend/src/app/pages/peliculas/peliculas.component.ts (fragmento)

```

export class PeliculasComponent implements OnInit {
    form!: FormGroup;
    peliculas: Película[] = [];
    generoConsulta = 'Drama';

    ngOnInit(): void {
        this.form = this.fb.group({
            titulo: ['', [Validators.required, Validators.minLength(2)]],
            anio: [new Date().getFullYear()],
            genero: ['Drama'],
        });
        this.cargarPeliculas();
    }

    agregarPelicula(): void {
        if (this.form.invalid || this.mongoUnavailable) return;
        const { titulo, anio, genero } = this.form.value;
        this.peliculaService.crear({ titulo, anio, genero }).subscribe({
            next: () => {

```



```

    this.mensaje = `${titulo}` añadida correctamente.`;
    this.cargarPelículas();
  },
  error: (err) => this.handleError(err),
});
}
}

```

```

+ selector: 'ui-button',
+ standalone: true,
+ imports: [CommonModule],
+ template: `
+   <button
+ ... truncated (43 more lines) · ctrl+r to review
Edited input.component.ts +55
+ import { Component, forwardRef, Input } from '@angular/core';
+ import { ControlValueAccessor, NG_VALUE_ACCESSOR } from '@angular/forms';
+ @Component({
+   selector: 'ui-input',
+   standalone: true,
+   providers: [
+     {
+       provide: NG_VALUE_ACCESSOR,
+       useExisting: forwardRef(() => UiInputComponent),
+       multi: true,
+     },
+   ],
+ })
+ ... truncated (44 more lines) · ctrl+r to review
Edited card.component.ts +61
+ import { Component } from '@angular/core';
+ import { CommonModule } from '@angular/common';
+ @Component({
+   selector: 'ui-card',
+   standalone: true,
+   imports: [CommonModule],
+   template: `
+     <div class="rounded-lg border bg-card text-card-foreground shadow-sm">
+       <ng-content></ng-content>
+     </div>
+   `,
+ })
+ ... truncated (50 more lines) · ctrl+r to review
Edited peliculas.component.ts +111
+ import { Component, OnInit } from '@angular/core';
+ import { CommonModule } from '@angular/common';
+ import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
+ import { Pelicula, PeliculaService } from '../services/pelicula.service';
+ import { UiButtonComponent } from '../ui/button.component';
+ import { UiInputComponent } from '../ui/input.component';
+ import {
+   UiCardComponent,
+   UiCardContentComponent,
+   UiCardDescriptionComponent,
+   UiCardHeaderComponent,
+   UiCardTitleComponent,
+ } from '../ui/card.component';
+ ... truncated (100 more lines) · ctrl+r to review
Edited peliculas.component.html +114
+ <div class="mx-auto max-w-4xl space-y-6 p-6">
+   <div class="space-y-2 text-center">
+     <h1 class="text-3xl font-bold tracking-tight">Catálogo de Películas</h1>
+     <p class="text-muted-foreground">
+       Añade nombres de películas a la lista. CRUD con Angular, Node.js y MongoDB.
+     </p>
+   </div>
+
+   @if (mensaje) {
+     <div class="rounded-md border border-green-200 bg-green-50 px-4 py-3 text-sm text-green-800">
+       {{ mensaje }}
+     </div>
+   }
+ ... truncated (103 more lines) · ctrl+r to review
Edited app.routes.ts +5 -1

```

Figura 4. Creación de los componentes de interfaz ui-button, ui-input (ControlValueAccessor) y ui-card, y de la página de películas.

Para cerrar la etapa se ajustaron los archivos globales del frontend: el componente raíz `app.component.ts` se convirtió a plantilla en línea con un encabezado de aplicación, el `index.html` se tradujo al español (atributo `lang="es"` y título "Cine UNETI — Catálogo de Películas") y se completó el `.gitignore` con las carpetas de artefactos (`node_modules/`, `dist/`, `.angular/`) y el archivo `.env`, para que las credenciales nunca se versionen. Estos cambios se aprecian en la figura 5.

```

Edited app.component.ts +17 -5
@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.component.html',
  styleUrls: ['./app.component.css']
  template: `
    <div class="min-h-screen bg-background">
      <header class="border-b">
        <div class="mx-auto flex max-w-4xl items-center justify-between px-6 py-4">
          <div class="flex items-center gap-2">
... truncated (15 more lines) · ctrl+r to review
Edited pelculas.component.ts +1 -1
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormBuilder, FormGroup, ReactiveFormsModule, Validators } from '@angular/forms';
import { FormBuilder, FormGroup, FormsModule, ReactiveFormsModule, Validators } from '@angular/forms';
import { Pelicula, PeliculaService } from '../services/pelicula.service';
import { UIButtonComponent } from '../ui/button.component';
import { UIInputComponent } from '../ui/input.component';

standalone: true,
imports: [
  CommonModule,
  FormsModule,
  ReactiveFormsModule,
  UIButtonComponent,
  UIInputComponent,
]
Read frontend/package.json
Edited index.html +2 -2
<!doctype html>
<html lang="en">
+ <html lang="es">
<head>
  <meta charset="utf-8">
  <title>Frontend</title>
  <title>Cine UNETI - Catálogo de Películas</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
Edited .gitignore +7 -1
+ node_modules/
+ dist/
+ .env
+ *.log
+ .DS_Store
+ .angular/
+ coverage/
Edited .env +3 -1
+ MONGODB_URI=mongodb://localhost:27017
+ MONGODB_DB=cine_uneti
+ PORT=3000
Edited db.js +1 -1

```

Figura 5. Ajustes globales del frontend: componente raíz con plantilla en línea, index.html en español y .gitignore con node_modules, dist y .env.

ETAPA 5

Orquestación de la Ejecución y Arquitectura de Tres Capas

La quinta etapa consolidó la experiencia de ejecución del sistema. Gracias a la herramienta concurrently, el comando `yarn start` ejecutado desde la raíz del monorepo levanta en paralelo los dos procesos con etiquetas de color distinguibles en la terminal: `api` (backend Node.js en `http://localhost:3000`) y `web` (Angular en `http://localhost:4200`). La figura 6 muestra el resumen final del montaje, con las cinco colecciones, las funcionalidades implementadas y la guía de ejecución en cuatro pasos.

Parte	Tecnología
Frontend	Angular 19 + Tailwind CSS + componentes estilo shadcn/ui
Backend	Node.js + Express + MongoDB Driver
Base de datos	MongoDB (cine_uneti)

5 colecciones MongoDB (4+ campos, 5+ documentos c/u)

- películas – título, año, género, duracionMinutos, calificacion
- directores – nombre, nacionalidad, películasDirigidas, premiosOscar
- generos – nombre, descripcion, popularidad, colorHex
- actores – nombre, edad, nacionalidad, películasDestacadas
- resenas – película, autor, calificacion, comentario

Funcionalidades

- Angular: formulario para añadir películas a una lista, verlas y eliminarlas
- CRUD completo en Node.js para las 5 colecciones (GET, POST, PUT, DELETE)
- Consulta sencilla: películas filtradas por género
- Scripts de seed: scripts/seed-db.sh (Linux) y scripts/seed-db.ps1 (Windows)
- Tras yarn install se ejecuta el seed automáticamente (postinstall)

Cómo ejecutarlo

```
# 1. Configurar entorno
cp .env.example .env
# 2. Tener MongoDB corriendo (local o Atlas)
# 3. Instalar (incluye seed)
yarn install
# 4. Iniciar todo
yarn start
```

• Frontend: <http://localhost:4200>
• API: <http://localhost:3000>

Si MongoDB no está activo durante yarn install, el seed se omite con un aviso. Puedes ejecutarlo después con yarn seed.

Para omitir el seed en install: SKIP_SEED=1 yarn install

Para tu entrega PDF

El README.md ya incluye descripción, instrucciones (git clone, yarn install, variables de entorno, yarn start) y documentación de la API. Solo te faltan:

1. Capturas de MongoDB Compass/Atlas con las 5 colecciones
2. Subir a GitHub y pegar el enlace en el PDF
3. Portada con tu nombre y logo UNETI

¿Quieres que ajuste algo? Por ejemplo: tu nombre en archivos, conexión a MongoDB Atlas en lugar de local, o una página extra de CRUD para las otras colecciones desde Angular.

me interesa que al escribir yarn start como comando, arranque los dos. Ahora, era necesario express para angular? esto es solo una pregunta.

Comprobando que yarn start levante backend y frontend, y respondiendo lo de Express.

Read 3 files
Read package.json
Read frontend/package.json
Read backend/package.json

yarn start en la raíz ya está configurado para levantar ambos. Voy a mejorar el script para que se vea más claro en la terminal.

Edited package.json +1 -1

Figura 6. Resumen del sistema montado: tecnologías por capa, cinco colecciones de MongoDB, funcionalidades y guía de ejecución con yarn start.

En esta fase se documentó además una aclaración conceptual importante para la asignatura: ¿era necesario Express si ya se usaba Angular? La respuesta es afirmativa, porque Angular no forma parte del servidor: corre en el navegador y solo consume la API por HTTP.

Express es quien atiende esas peticiones y gestiona la conexión con la base de datos; sin él (o un equivalente como Fastify o NestJS), no habría servidor que ejecutara las operaciones CRUD contra MongoDB. El sistema queda así organizado en tres capas independientes: Angular como interfaz, Express como API y MongoDB como almacén de datos, comunicadas por HTTP y por el controlador oficial de MongoDB, respectivamente.

El comportamiento de la API puede verificarse directamente con peticiones HTTP. Los siguientes ejemplos, incluidos también en el README del repositorio, crean una película y ejecutan la consulta sencilla por género.

Verificación de la API con curl

```
# Crear película
curl -X POST http://localhost:3000/api/peliculas \
  -H "Content-Type: application/json" \
  -d '{"titulo":"Inception","anio":2010,"genero":"Ciencia Ficción}'

# Consulta sencilla por género
curl http://localhost:3000/api/peliculas/consulta/genero/Drama
```

```

"seed": "node scripts/run-seed.js"
yarn start -- los dos a la vez

Eso ya está configurado en la raíz del proyecto (no dentro de frontend/):

cd informatica-basededatos # carpeta raíz
yarn start

Levanta en paralelo:
• api → backend Node.js en http://localhost:3000
• web → Angular en http://localhost:4200

Importante: hay que ejecutarlo desde la raíz, donde está el package.json principal. Si corres yarn start dentro de frontend/, solo arranca Angular.

¿Era necesario Express para Angular?

No. Express no es parte de Angular.



| Pieza   | Qué usa                                        |
|---------|------------------------------------------------|
| Angular | Su propio servidor de desarrollo (ng serve)    |
| Express | El backend → la API REST que habla con MongoDB |



Angular corre en el navegador y llama por HTTP a Express (http://localhost:3000/api/...). Express lo pide la tarea: "CRUD en Node.js con conexión a MongoDB".

Sin Express (o algo equivalente como Fastify o NestJS) no habría servidor que gestione la base de datos; Angular por sí solo no se conecta a MongoDB de forma directa en producción.

En resumen: Angular = interfaz, Express = API, MongoDB = datos. Son tres capas distintas.

tengo gh instalado, por favor verifica y crea un repositorio llamado angular-crud-uneti y sube este código para allá.

Verificando gh y el estado del proyecto antes de crear el repositorio y subir el código.

$ gh --version && gh auth status 2>&1 1.1s
  6 output lines hidden: ctrl+o to expand
  - To re-authenticate, run: gh auth login -h github.com
  - To forget about this account, run: gh auth logout -h github.com -u miguelfiguera

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && git status 2>&1; ls -la 560ms
drwxrwxr-x 1 root root 64 jul 12 19:17 scripts
-rw-rw-r-- 1 root root 364487 jul 12 19:18 yarn.lock

El token de gh parece inválido; comprobando si funciona con permisos completos.

$ gh auth status 2>&1; git config --global credential.helper 2>/dev/null; ls -la ~/.config/gh/ 2>/dev/null
Waiting for approval...

$ gh auth status 2>&1; git config --global credential.helper 2>/dev/null; ls -la ~/.config/gh/ 2>/dev/null in .
Not in allowlist: gh, git config
→ Run (once) (y)
  Add Shell(gh), Shell(git config) to allowlist? (tab)
  Run Everything (shift+tab)
  Skip (esc or n)
  
```

Figura 7. Aclaración de la arquitectura de tres capas (Angular, Express, MongoDB) y verificación de la autenticación de GitHub CLI.

ETAPA 6

Publicación del Repositorio en GitHub

La sexta etapa publicó el código fuente en GitHub para respaldar el trabajo y hacerlo accesible a la evaluación docente. Con la interfaz de línea de comandos gh se verificó primero el estado de la autenticación (`gh auth status`), comprobando que la cuenta miguelfiguera contaba con credenciales válidas y permisos suficientes. La figura 8 documenta esta verificación previa.

el nombre angular-crud-uneti y la rama principal main quedó enlazada al remoto. El archivo .env no se subió por estar excluido en .gitignore, práctica correcta de manejo de secretos. La figura 9 muestra la secuencia completa y la dirección final del repositorio.

Secuencia de publicación en GitHub

```
gh --version && gh auth status
git init
git add .
git commit -m "Proyecto CRUD Angular + Node.js + MongoDB para UNETI."
# rama main enlazada al remoto y código publicado
gh repo view miguelfiguera/angular-crud-uneti \
  --json url,visibility,defaultBranchRef
# → https://github.com/miguelfiguera/angular-crud-uneti (público, main)
```

```

tarea: "CRUD en Node.js con conexión a MongoDB".
Sin Express (o algo equivalente como Fastify o NestJS) no habría servidor que gestione la base de datos;
Angular por sí solo no se conecta a MongoDB de forma directa en producción.

En resumen: Angular = interfaz, Express = API, MongoDB = datos. Son tres capas distintas.

tengo gh instalado, por favor verifica y crea un repositorio llamado angular-crud-uneti y sube este
codigo para alla.

Verificando gh y el estado del proyecto antes de crear el repositorio y subir el código.

$ gh --version && gh auth status 2>&1 1.1s
... 6 output lines hidden · ctrl+o to expand
- To re-authenticate, run: gh auth login -h github.com
- To forget about this account, run: gh auth logout -h github.com -u miguelfiguera

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && git status 2>&1; ls -la 560ms
... 14 output lines hidden · ctrl+o to expand
drwxrwxr-x 1 root root 64 jul 12 19:17 scripts
-rw-rw-r-- 1 root root 364487 jul 12 19:18 yarn.lock

El token de gh parece inválido; comprobando si funciona con permisos completos.

$ gh auth status 2>&1; git config --global credential.helper 2>/dev/null; ls -la ~/.config/gh/ 2>/dev/null 6m
... 9 output lines hidden · ctrl+o to expand
-rw----- 1 chepino chepino 902 abr 18 14:16 config.yml
-rw----- 1 chepino chepino 92 abr 18 14:16 hosts.yml

Autenticación confirmada. Inicializando git, creando el commit y subiendo el repositorio.

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && git init && git add . && git status 833ms
... 58 output lines hidden · ctrl+o to expand
new file:   yarn.lock

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && git commit --trailer "Co-authored-by: Cursor
<cursoragent@cursor.com>" -m "$(cat <<'EOF'
Proyecto CRUD Angular + Node.js + MongoDB para UNETI. 13s
... 4 input + 56 output lines hidden · ctrl+o to expand
* [new branch] HEAD -> main
branch 'main' set up to track 'origin/main'.

$ gh repo view miguelfiguera/angular-crud-uneti --json url,visibility,defaultBranchRef 2>&1 && cd
/home/chepino/repos/Uneti-related/informatica-basededatos && git status 21s
... 7 output lines hidden · ctrl+o to expand
nothing added to commit but untracked files present (use "git add" to track)

Listo. Verifiqué gh (v2.46.0, cuenta miguelfiguera autenticada) y el código ya está en GitHub.

Repositorio: https://github.com/miguelfiguera/angular-crud-uneti
Visibilidad: público
Rama: main

Se subieron 52 archivos, incluyendo backend, frontend, scripts de seed, README y las fotos de Fotos/ (1-7).
El archivo .env no se subió (está en .gitignore).

Hay una foto nueva sin subir: Fotos/8.png. Si quieres incluirla, dímelo y la subo en otro commit.

→ Add a follow-up

Composer 2.5 Fast · 28.8% · 31 files edited
~/repos/Uneti-related/informatica-basededatos · main

```

Figura 9. Publicación del repositorio angular-crud-uneti: git init, commit inicial con 52 archivos y rama main enlazada al remoto público.

ETAPA 7

Verificación End-to-End con Playwright

La séptima etapa validó el funcionamiento integral del sistema mediante un script de pruebas end-to-end escrito con Playwright (`scripts/e2e-test.cjs`). Antes de las pruebas se preparó el entorno: se inició MongoDB en `localhost:27017`, se ejecutó el seed que creó las cinco colecciones con cinco documentos cada una, y se levantó la aplicación completa con `yarn start`. El script ejerce primero la API con `fetch` —verificación de salud, listado de películas, consulta por género y creación de registros— y luego automatiza el navegador Chromium para probar la interfaz real: carga de la página, visualización de las películas del seed, adición y eliminación de una película y ejecución de la consulta por género desde el formulario.

scripts/e2e-test.cjs (fragmento)

```
const { chromium } = require('playwright');

const API = 'http://localhost:3000';
const APP = 'http://localhost:4200';
const TEST_MOVIE = `Test Película E2E ${Date.now()}`;

async function run() {
  const results = [];
  const pass = (name, detail = '') => results.push({ name, ok: true, detail });
  const fail = (name, detail = '') => results.push({ name, ok: false, detail });

  // --- Pruebas de API ---
  const health = await fetch(`${API}/api/health`).then((r) => r.json());
  if (health.status === 'ok') pass('API health check', health.message);

  const peliculas = await fetch(`${API}/api/peliculas`).then((r) => r.json());
  if (Array.isArray(peliculas) && peliculas.length >= 5) {
    pass('API listar películas', `${peliculas.length} películas en MongoDB`);
  }

  const res = await fetch(`${API}/api/peliculas/consulta/genero/Drama`)
    .then((r) => r.json());
  if (res.total >= 1 && Array.isArray(res.peliculas)) {
    pass('API consulta por género', `${res.total} película(s) Drama`);
  }
  // ... luego Chromium abre http://localhost:4200 y prueba la interfaz
}
```

```

const { chromium } = require('playwright');

const API = 'http://localhost:3000';
const APP = 'http://localhost:4200';
const TEST_MOVIE = `Test Película E2E ${Date.now()}`;

async function run() {
  const results = [];
  const pass = (name, detail = '') => results.push({ name, ok: true, detail });
  const fail = (name, detail = '') => results.push({ name, ok: false, detail });

  // --- API tests ---
  ... truncated (124 more lines) ... ctrl+r to review
}

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && npx -y playwright@1.51.0 install chromium
2>&1 && node scripts/e2e-test.mjs 2>&1 exit 1 * 52s
... 86 output lines hidden ... ctrl+o to expand
Node.js v25.2.0

Deleted scripts/e2e-test.mjs
Completed

Edited e2e-test.cjs +133

const { chromium } = require('playwright');

const API = 'http://localhost:3000';
const APP = 'http://localhost:4200';
const TEST_MOVIE = `Test Película E2E ${Date.now()}`;

async function run() {
  const results = [];
  const pass = (name, detail = '') => results.push({ name, ok: true, detail });
  const fail = (name, detail = '') => results.push({ name, ok: false, detail });

  try {
    ... truncated (122 more lines) ... ctrl+r to review
  }
}

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && PLAYWRIGHT_BROWSERS_PATH=/tmp/cursor-sandbox-cache/68e920f49c146b97c3178b14e7abec0b/playwright node -e "require('playwright');" 2>&1 || yarn add -D playwright@1.51.0 2>&1;
PLAYWRIGHT_BROWSERS_PATH=/tmp/cursor-sandbox-cache/68e920f49c146b97c3178b14e7abec0b/playwright node scripts/e2e-test.cjs 2>&1 exit 1 * 636ms
... 49 output lines hidden ... ctrl+o to expand
Node.js v25.2.0

$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && yarn add -W -D playwright@1.51.0 2>&1 && PLAYWRIGHT_BROWSERS_PATH=/tmp/cursor-sandbox-cache/68e920f49c146b97c3178b14e7abec0b/playwright node scripts/e2e-test.cjs 2>&1 9.3s
... 43 output lines hidden ... ctrl+o to expand
9/9 pruebas pasaron

$ mongosh cine_uneti --quiet --eval "db.getCollectionNames().forEach(c => print(c + ': ' + db[c].countDocuments()))" 669ms
... 3 output lines hidden ... ctrl+o to expand
actores: 5
películas: 5

Todo verificado y funcionando. Resumen:

Preparación
1. MongoDB no estaba corriendo -> lo inicié en localhost:27017
2. Seed ejecutado -> 5 colecciones con 5 documentos cada una
3. yarn start activo -> API en :3000 y Angular en :4200

Nota: Playwright MCP no está conectado en esta sesión de Cursor, así que usé Playwright CLI (mismo motor de browser) con un script E2E.

Resultados: 9/9 pruebas pasaron
  
```

Añadir película
Escribe el nombre de la película y opcionalmente el año y

Nombre de la película
Ej: Inception

Lista de películas
5 película(s) registrada(s) en MongoDB

El Padrino	1972 · Drama · ★ 9.2
1994 · Crimen · ★ 8.9	
El Señor de los Anillos	2001 · Fantasía · ★ 8.8
Matrix	1999 · Ciencia Ficción · ★ 8.7
Parásitos	2019 · Drama · ★ 8.6

Consulta sencilla
Buscar películas por género en MongoDB

Figura 10. Script de pruebas end-to-end e2e-test.cjs con Playwright y ejecución con el navegador Chromium instalado.

El resultado fue de nueve pruebas superadas de nueve ejecutadas: cuatro contra la API y cinco contra el frontend. Como verificación adicional se consultó MongoDB directamente con mongosh, confirmando el conteo de documentos por colección: películas, directores, generos,

actores y resenas con cinco documentos cada una. La figura 11 recoge la tabla completa de resultados y el estado final del sistema en ejecución.

The screenshot shows a terminal window with the following content:

```
$ cd /home/chepino/repos/Uneti-related/informatica-basededatos && yarn add -W -D playwright@1.51.0 2>&1 &&
PLAYWRIGHT_BROWSERS_PATH=/tmp/cursor-sandbox-cache/68e920f49c146b97c3178b14e7abec0b/playwright node
scripts/e2e-test.cjs 2>&1 9.3s
... 43 output lines hidden · ctrl+o to expand
9/9 pruebas pasaron
```

Below the terminal output, there is a summary of the test results:

Todo verificado y funcionando. Resumen:

Preparación

1. MongoDB no estaba corriendo → lo inicié en localhost:27017
2. Seed ejecutado → 5 colecciones con 5 documentos cada una
3. yarn start activo → API en :3000 y Angular en :4200

Nota: Playwright MCP no está conectado en esta sesión de Cursor, así que usé Playwright CLI (mismo motor de browser) con un script E2E.

Resultados: 9/9 pruebas pasaron

Prueba	Resultado
API health check	✓
API listar películas	✓ 5 en MongoDB
API consulta por género (Drama)	✓ 2 películas
API crear película	✓
Frontend carga página principal	✓ "Catálogo de Películas"
Frontend muestra películas seed	✓ 5 registradas
Frontend añadir película	✓
Frontend eliminar película	✓
Frontend consulta por género	✓ 2 resultados

MongoDB – colecciones

```
películas: 5
directores: 5
generos: 5
actores: 5
resenas: 5
```

Estado actual

- MongoDB corriendo en background
- App corriendo (yarn start) – puedes abrir <http://localhost:4200>
- Script de prueba guardado en scripts/e2e-test.cjs (para repetir: node scripts/e2e-test.cjs)

Si quieres usar Playwright MCP directamente desde Cursor, hay que habilitarlo en la configuración MCP del proyecto. ¿Quieres que lo configure o que suba el script E2E al repo de GitHub?

agrega darkmode y ponlo por defecto por favor.

Revisando la configuración actual de estilos y del componente principal para activar dark mode por defecto.

Read 5 files
... 2 earlier items hidden

The background image shows a web application interface titled "Catálogo de Películas". It includes a search bar, a list of movies (e.g., Inception, Pulp Fiction, El Señor de los Anillos, Matrix), and a sidebar with filters like "Género" and "Drama".

Figura 11. Resultados de la verificación: nueve pruebas superadas de nueve, y conteo de documentos por colección confirmado con mongosh.

ETAPA 8

Modo Oscuro por Defecto y Cierre del Proyecto

La octava y última etapa incorporó el modo oscuro como tema visual de la aplicación. En el `index.html` se añadió la clase `dark` al elemento raíz y la metaetiqueta `color-scheme`, junto con un script en línea que aplica el tema antes del primer renderizado para evitar el destello de pantalla clara al cargar. Se creó además un servicio de tema que persistía la preferencia del usuario en `localStorage`, con un botón alternador en el encabezado; en la revisión final del proyecto se decidió fijar el modo oscuro como único tema de la interfaz.

frontend/src/index.html (estado final)

```
<!doctype html>
<html lang="es" class="dark">
<head>
  <meta charset="utf-8">
  <title>Cine UNETI - Catálogo de Películas</title>
  <base href="/">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <meta name="color-scheme" content="dark">
  <link rel="icon" type="image/x-icon" href="favicon.ico">
</head>
<body>
  <app-root></app-root>
</body>
</html>
```

El ajuste implicó también refinar las variables CSS del tema oscuro en `styles.css` —elevando ligeramente la luminosidad de las tarjetas para mantener el contraste— y añadir variantes `dark`: de Tailwind a las alertas de éxito y de error de la página de películas, de modo que los mensajes conserven legibilidad en ambos esquemas de color. Tras compilar el frontend con `yarn build` y verificar el resultado en el navegador, el cambio se registró en el historial del repositorio con el commit "dark mode only", cerrando así el ciclo de desarrollo del sistema. La figura 12 documenta estos últimos ajustes.

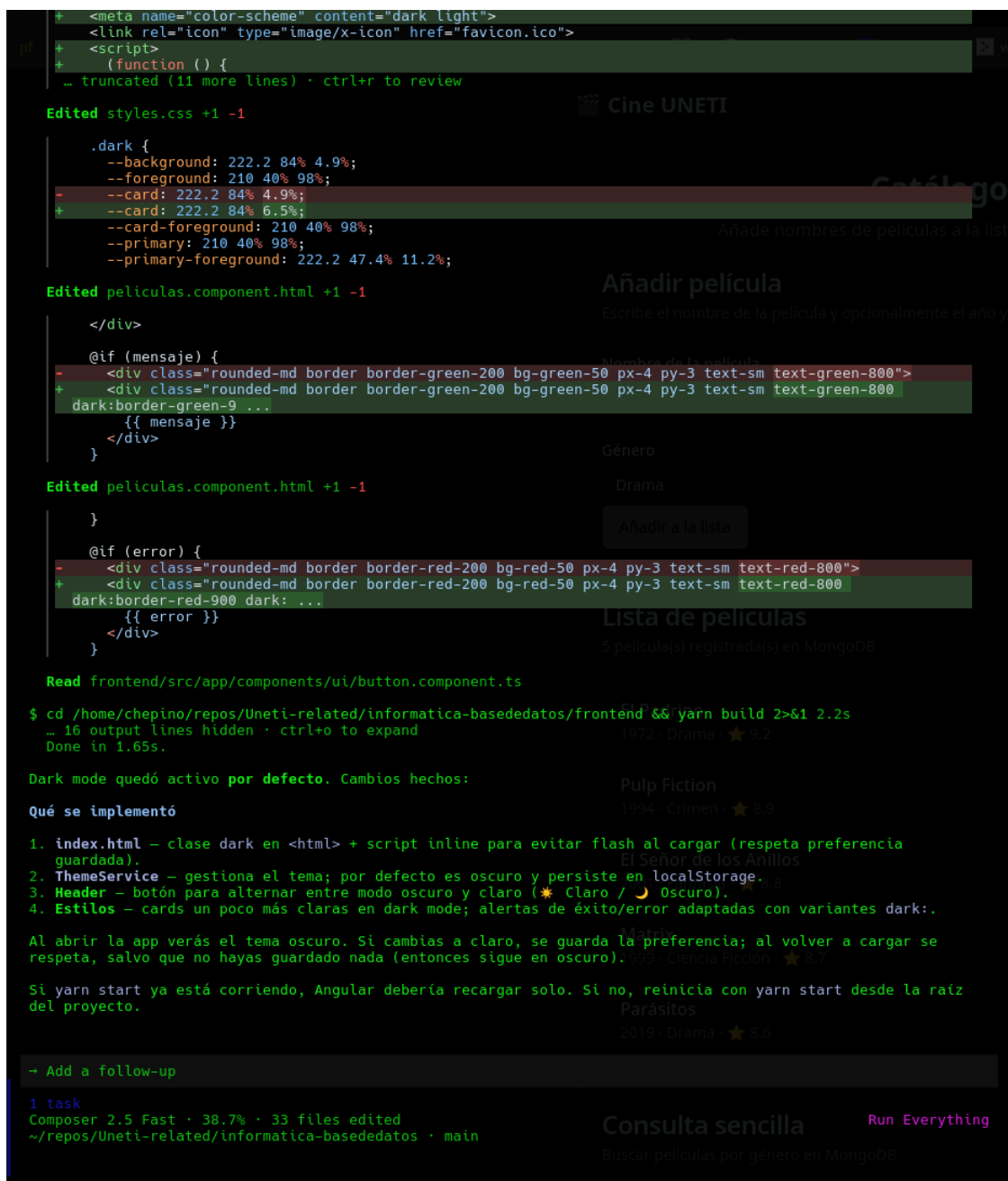


Figura 12. Implementación del modo oscuro: variables CSS del tema, variantes dark: en las alertas y compilación final con yarn build.

CONCLUSIÓN

El desarrollo del sistema CRUD de películas permitió aplicar de manera integrada los contenidos del módulo Informática — Bases de Datos: el diseño de una base de datos documental con cinco colecciones en MongoDB, la construcción de una API REST con Node.js y Express que expone las operaciones de creación, lectura, actualización y eliminación, y el consumo de esos servicios desde una interfaz Angular con formularios reactivos y componentes reutilizables. Las ocho etapas descritas —montaje del monorepo, backend, scripts de seed, frontend, orquestación, publicación, verificación y tematización— conforman un procedimiento sistemático y replicable para proyectos de similar naturaleza.

Desde el punto de vista técnico, el proyecto dejó aprendizajes concretos: la separación en tres capas (interfaz, API y datos) clarifica responsabilidades y facilita el mantenimiento; el enrutador CRUD genérico con lista blanca de colecciones evita la duplicación de código sin sacrificar seguridad; los scripts de seed multiplataforma con ejecución automática tras la instalación reducen la fricción de puesta en marcha; y las pruebas end-to-end con Playwright, con nueve verificaciones superadas, aportan evidencia objetiva de que el sistema funciona de extremo a extremo. Las doce capturas de pantalla conservadas durante el proceso constituyen la evidencia verificable de cada fase, mientras que el repositorio público en GitHub garantiza la reproducibilidad y permanencia del producto final.

REFERENCIAS

- American Psychological Association. (2020). Publication Manual of the American Psychological Association (7ma ed.). <https://doi.org/10.1037/0000165-000>
- Angular Team. (2025). Angular Documentation. <https://angular.dev/>
- Express Team. (2025). Express - Node.js web application framework. <https://expressjs.com/>
- Figuera, M. (2026). angular-crud-uneti [Código fuente]. GitHub.
<https://github.com/miguelfiguera/angular-crud-uneti>
- Microsoft. (2025). Playwright: Fast and reliable end-to-end testing. <https://playwright.dev/>
- MongoDB, Inc. (2025). MongoDB Manual. <https://www.mongodb.com/docs/manual/>
- OpenJS Foundation. (2025). Node.js Documentation. <https://nodejs.org/docs/>
- Tailwind Labs. (2025). Tailwind CSS Documentation. <https://tailwindcss.com/docs>

ANEXOS

Anexo A. Repositorio del código fuente

El código fuente completo del sistema —backend, frontend, scripts de seed, pruebas end-to-end y el generador del presente documento— se encuentra publicado con visibilidad pública en el siguiente repositorio de GitHub. Para reproducir el proyecto basta con clonarlo, disponer de Node.js 18 o superior, Yarn y una instancia de MongoDB, y ejecutar `yarn install` seguido de `yarn start`.

URL: <https://github.com/miguelfiguera/angular-crud-uneti>

Anexo B. Estructura del proyecto

La siguiente estructura resume la organización de carpetas del monorepo, con la separación entre la API, la interfaz y los scripts de apoyo descrita a lo largo del documento.

Estructura de carpetas

```

├── backend/                # API Node.js + Express + MongoDB
│   └── src/
│       ├── config/        # Conexión a MongoDB (db.js)
│       ├── routes/        # Router CRUD genérico
│       └── seed/          # Poblado de las 5 colecciones
├── frontend/              # Angular 19 + Tailwind CSS
│   └── src/app/
│       ├── components/ui/  # button, input, card (estilo shadcn)
│       ├── pages/peliculas/ # página CRUD principal
│       └── services/       # PeliculaService (HttpClient)
├── scripts/
│   ├── seed-db.sh         # Seed Linux/macOS
│   ├── seed-db.ps1        # Seed Windows
│   ├── run-seed.js        # Orquestador postinstall
│   └── e2e-test.cjs       # Pruebas end-to-end (Playwright)
├── docs/                  # Este documento y su generador
├── Fotos/                 # Capturas del proceso (1.png ... 12.png)
└── README.md              # Instrucciones de ejecución

```